Session 4 — Filters & Convolution (everything coded here)

In [1]:
```
import sys
import time
from pathlib import Path

import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from scipy import ndimage

# this line just walks up the folders until it finds the repo, where I have
some configuration and the data
ROOT = next(p for p in [Path.cwd().resolve(), *Path.cwd().resolve().parents]
            if (p / "common" / "config.py").is_file())
sys.path.insert(0, str(ROOT))

from common import config, milk10k      # data loading only: image paths +
metadata

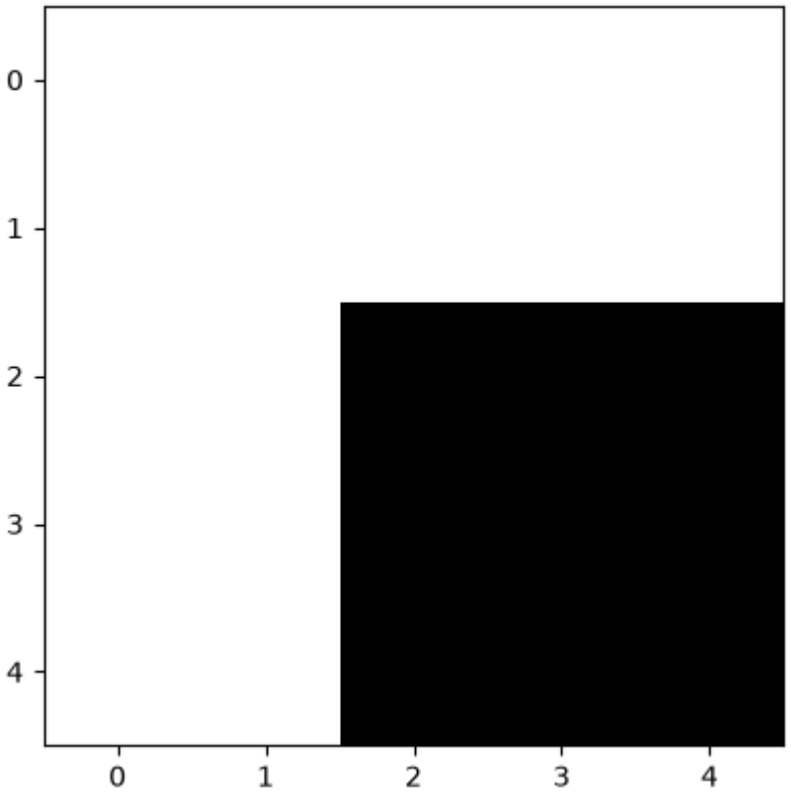
np.set_printoptions(precision=1, suppress=True)
```

In [2]:
```
image = np.array([
    [100, 100, 100, 100, 100],
    [100, 100, 100, 100, 100],
    [100, 100, 10,  10,  10 ],
    [100, 100, 10,  10,  10 ],
    [100, 100, 10,  10,  10 ],
],dtype=float)
image
```

Out[2]:
```
array([[100., 100., 100., 100., 100.],
       [100., 100., 100., 100., 100.],
       [100., 100.,  10.,  10.,  10.],
       [100., 100.,  10.,  10.,  10.],
       [100., 100.,  10.,  10.,  10.]])
```

In [3]:
```
plt.figure()
plt.imshow(image,cmap='gray')
```

Out[3]:
```
<matplotlib.image.AxesImage at 0x1988657c6e0>
```



In [4]:
```
# Box blur kernel of size 3x3
kernel = np.ones((3,3)) / 9
print(kernel)
print("sum of weights of the box blur kernel: ", kernel.sum())
```

```
[[0.1 0.1 0.1]
 [0.1 0.1 0.1]
 [0.1 0.1 0.1]]
sum of weights of the box blur kernel:  1.0
```

In [5]:
```
#top left corner of the window
row, col = 1,1
patch = image[row:row+3, col:col+3]    # the 9 pixels under the window
products = patch * kernel              # cell by cell multiplication
total = products.sum()
print(patch)
print(products)
print(total)
```

```
[[100. 100. 100.]
 [100.  10.  10.]
 [100.  10.  10.]]
[[11.1 11.1 11.1]
 [11.1  1.1  1.1]
 [11.1  1.1  1.1]]
60.0
```

In [6]:
```
def conv_step(image, kernel, row, col):
    """ one window: patch -> products -> sum"""
    k = kernel.shape[0]
    patch = image[row:row+k, col:col+k]
    products = patch * kernel
    return patch, products, products.sum()
```

```
# position of the window (0,0)
conv_step(image, kernel, 0, 0)
```

Out[6]:
```
(array([[100., 100., 100.],
        [100., 100., 100.],
        [100., 100.,  10.]]),
 array([[11.1, 11.1, 11.1],
        [11.1, 11.1, 11.1],
        [11.1, 11.1,  1.1]]),
 np.float64(90.0))
```

In [7]:
```
# SLIDING THE KERNEL
# 5x5 input image
# 3x3 kernel
# NO padding
# Stride of 1
# -> 3x3

n_out = image.shape[0] - kernel.shape[0] + 1
out = np.zeros((n_out, n_out))

for r in range(n_out):
    for c in range(n_out):
        _, _, out[r,c] = conv_step(image, kernel, r, c)
        print(f"window at position {r}, {c} -> {out[r,c]}")

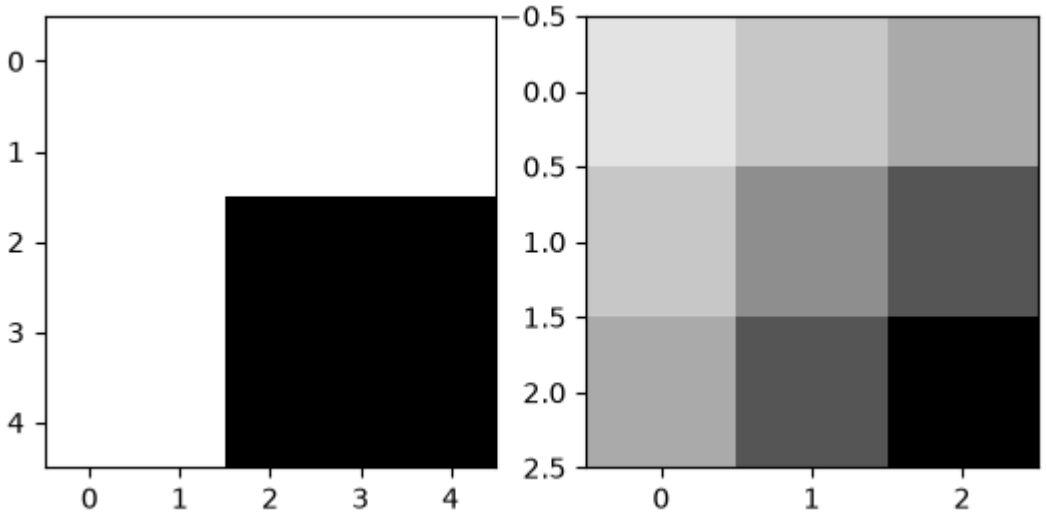
out
```

```
window at position 0, 0 -> 90.0
window at position 0, 1 -> 80.0
window at position 0, 2 -> 70.0
window at position 1, 0 -> 80.0
window at position 1, 1 -> 60.0
window at position 1, 2 -> 40.0
window at position 2, 0 -> 70.0
window at position 2, 1 -> 40.0
window at position 2, 2 -> 10.0
```

Out[7]:
```
array([[90., 80., 70.],
       [80., 60., 40.],
       [70., 40., 10.]])
```

In [8]:
```
plt.subplot(1,2,1)
plt.imshow(image,'gray',vmin=10, vmax=100)
plt.subplot(1,2,2)
plt.imshow(out, cmap='gray', vmin=10, vmax=100)
```

Out[8]:
```
<matplotlib.image.AxesImage at 0x198887d02d0>
```



Padding and stride

In [9]:
```
def output_size(n, k, padding=0, stride=1):
    return (n + 2*padding - k) // stride + 1

# input 5x5; k 3x3, padding 0, stride 2
print(output_size(5,3,0,2))
```

```
2
```

conv2d

In [10]:
```
def conv2d(image, kernel, padding, stride, pad_mode="constant"):
    k = kernel.shape[0]
    padded = np.pad(image, padding, mode=pad_mode)
    out_h = output_size(image.shape[0], k, padding, stride)
    out_w = output_size(image.shape[1], k, padding, stride)
    out = np.zeros((out_h, out_w))
    for r in range(out_h):
        for c in range(out_w):
            _, _, out[r,c] = conv_step(padded, kernel, r*stride, c*stride)
    return out

conv2d(image, kernel, padding=0, stride=1)
```

Out[10]:
```
array([[90., 80., 70.],
       [80., 60., 40.],
       [70., 40., 10.]])
```

In [11]:
```
conv2d(image, kernel, padding=0, stride=2)
```

Out[11]:
```
array([[90., 70.],
       [70., 10.]])
```

Different kernels

In [12]:
```
KERNELS = {
    "identity": np.array([[0, 0, 0], [0, 1, 0], [0, 0, 0]], dtype=float),
    "box_blur": np.ones((3, 3)) / 9,
    "gaussian": np.array([[1, 2, 1], [2, 4, 2], [1, 2, 1]]) / 16,
    "sharpen":  np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]], dtype=float),
```

```
    "outline":  np.array([[-1, -1, -1], [-1, 8, -1], [-1, -1, -1]],
dtype=float),
    "emboss":   np.array([[-2, -1, 0], [-1, 1, 1], [0, 1, 2]], dtype=float),
    "sobel_x":  np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=float),
    "sobel_y":  np.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]], dtype=float),
}
for name, k in KERNELS.items():
    print(f"{name:9s} sum = {k.sum():g}")

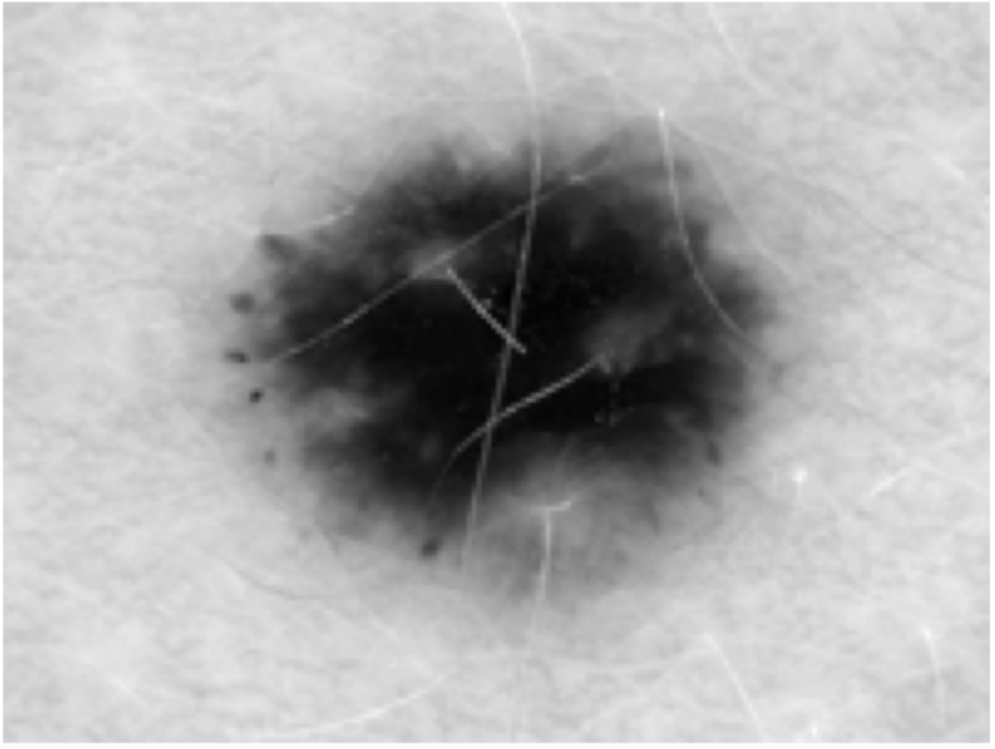
# Lesion_id -> "ISIC_0073863"
# plot 4cols x 2 rows
```

```
identity  sum = 1
box_blur  sum = 1
gaussian  sum = 1
sharpen   sum = 1
outline   sum = 0
emboss    sum = 1
sobel_x   sum = 0
sobel_y   sum = 0
```

In [13]:

```
isic_id = "ISIC_0073863"
img = Image.open(config.image_path(isic_id)).convert("L") # "L" = grayscale
img = img.resize((200, 150))
gray = np.array(img, dtype=float) # float: no overflow, negatives allowed
print(gray.shape, gray.min(), gray.max()) # (150, 200) 17.0 182.0
plt.imshow(gray, cmap="gray"); plt.axis("off");
```

```
(150, 200) 17.0 182.0
```



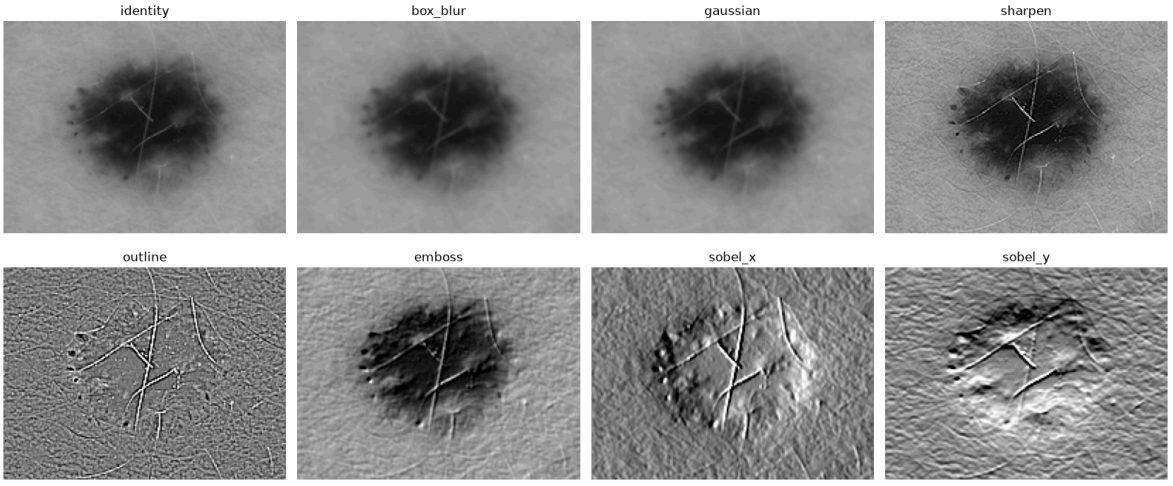
In [14]:

```
"""
`fast_conv2d`: scipy's `correlate` does exactly what our `conv2d` does, with
mirror padding, but in optimized C. We'll prove they're identical in a moment,
and measure the speed."
"""
def fast_conv2d(image, kernel):
    """Same as conv2d(..., padding=1, pad_mode="symmetric") but in optimized
C."""
```

```
    return ndimage.correlate(image, kernel, mode="reflect")

"""
 `stretch`, for display only. Edge outputs have negative values and a few huge
ones. If we map min to max, almost everything ends up the same grey. So we
stretch between the 1st and the 99th percentile instead, and clip the rest."
 """
def stretch(x, pct=1):
    """Contrast-stretch between the 1st and 99th percentile -> [0, 1] (for edge
maps)."""
    lo, hi = np.percentile(x, [pct, 100 - pct])
    return np.clip((x - lo) / (hi - lo), 0, 1)
```

In [15]:
```
fig, axes = plt.subplots(2, 4, figsize=(16, 7))
for ax, (name, k) in zip(axes.ravel(), KERNELS.items()):
    result = fast_conv2d(gray, k)
    if np.isclose(k.sum(), 1):
        ax.imshow(np.clip(result, 0, 255), cmap="gray", vmin=0, vmax=255)   #
keep brightness
    else:
        ax.imshow(stretch(result), cmap="gray")                              #
stretch edges
    ax.set_title(name); ax.axis("off")
plt.tight_layout()
```



In [16]:
```
t0 = time.perf_counter()
slow = conv2d(image, KERNELS["box_blur"], padding=1, stride=1,
pad_mode="symmetric")
t_slow = time.perf_counter() - t0
t0 = time.perf_counter()
fast = fast_conv2d(image, KERNELS["box_blur"])
t_fast = time.perf_counter() - t0
print(f"loops {t_slow:.5f}s | scipy {t_fast:.5f}s | {t_slow / t_fast:.5f}x
faster")
print("same result:", np.allclose(slow, fast))
```

```
loops 0.00039s | scipy 0.00018s | 2.21525x faster
same result: True
```